

OPENCLAW

1. Kiến trúc tổng thể

OpenClaw là một **self-hosted, multi-channel gateway for AI agents**. Trung tâm của toàn bộ hệ thống là một **Gateway process** chạy lâu dài, giữ vai trò “source of truth” cho sessions, routing và channel connections. Các bề mặt khác như CLI, web dashboard, macOS app, TUI và các kênh chat bên ngoài không tự giữ trạng thái agent riêng, mà đều nói chuyện với Gateway. Cách thiết kế này cho thấy OpenClaw là một **stateful runtime platform** hơn là một chatbot app đơn giản. Nó chạy trên **Node.js**, hỗ trợ **macOS, Linux và Windows/WSL2**, và dùng pnpm trong luồng build/dev chính thức.

- **Gateway-centric architecture**
Gateway là tiến trình trung tâm quản lý session state, channel bindings, routing và agent execution. Điều này giúp trạng thái hội thoại nhất quán hơn giữa nhiều surfaces.
- **Multi-channel by design**
OpenClaw được thiết kế để kết nối nhiều kênh như WhatsApp, Telegram, Discord, iMessage và các plugin channels khác, thay vì chỉ phục vụ một giao diện chat duy nhất.
- **Stateful agent runtime**
Agent không hoạt động theo kiểu request-response ngắn hạn. Nó có session, memory, compaction và transcript persistence để duy trì continuity qua thời gian.
- **Node/TypeScript platform**
README chính thức cho thấy OpenClaw chạy bằng Node.js, hỗ trợ cài qua npm/pnpm/bun, phù hợp với một codebase TypeScript/JavaScript hiện đại và dễ mở rộng bằng plugin.
- **Session + memory là trục kiến trúc chính**
Kiến trúc của OpenClaw xoay quanh việc giữ session bền vững, nén hội thoại cũ bằng compaction, và bổ sung long-term memory qua semantic retrieval. Đây là phần làm nên “agent continuity” của hệ thống.

2. Cách tổ chức mã nguồn

Về cấu trúc mã nguồn, OpenClaw được tổ chức như một codebase lớn theo kiểu monorepo, với các vùng chức năng tách khá rõ. Ở root repo có các thư mục như `src/`, `extensions/`, `apps/`, `packages/`, `skills/`, `docs/`, `scripts/`, cùng nhiều file cấu hình test riêng cho unit, gateway và extensions. Bên trong `src/` lại chia thành nhiều subsystem như `agents`, `channels`, `cli`, `commands`, `browser`, `bindings`, `bootstrap`, `auto-reply`, `canvas-host`, `acp`. Điều này cho thấy team không dồn toàn bộ runtime vào một lõi khó đọc, mà chia theo domain trách nhiệm tương đối rõ ràng.

- **src/ là vùng lõi runtime**
Đây là nơi chứa các thành phần quan trọng nhất như agent logic, channel handling, CLI entrypoints, browser/system interaction và bootstrap runtime.
- **extensions/ là vùng adapter/mở rộng**
Sự tồn tại riêng của thư mục này cho thấy các channel/provider/tool extensions được tách khỏi core, giúp mở rộng hệ sinh thái mà không cần sửa sâu vào lõi.
- **skills/ là lớp hướng dẫn cho model**
Theo docs, skills không chỉ là code executable mà còn là lớp hướng dẫn để model biết dùng tool nào và dùng như thế nào. Đây là một lựa chọn thiết kế rất đặc trưng của OpenClaw.

- **docs/ rất quan trọng để hiểu code**

OpenClaw có docs kỹ thuật khá chi tiết về architecture, sessions, compaction, memory, plugins, models. Với codebase kiểu này, đọc docs trước sẽ hiệu quả hơn nhảy thẳng vào source.

- **Entrypoint nên đọc từ CLI/Gateway trước**

README và docs đều xoay quanh các lệnh như onboarding, gateway, dashboard, models, channels. Vì vậy khi đọc repo, nên bắt đầu từ CLI/program và bootstrap flow trước, rồi mới xuống session manager, plugins và extensions.

- **Session persistence được tổ chức thành hai lớp**

Docs mô tả rõ sessions.json lưu metadata phiên, còn <sessionId>.jsonl lưu transcript append-only. Tức là phần state của hệ thống đã được tổ chức riêng thành “metadata layer” và “conversation layer”.

- **Model/provider layer cũng là một subsystem riêng**

OpenClaw không hard-code vào một model vendor; docs cho thấy có model discovery, auth profiles, provider catalogs và failover logic. Điều này ngụ ý trong mã nguồn có một lớp registry/routing khá độc lập cho providers.

3. Chất lượng code: điểm mạnh

Điểm mạnh lớn nhất của OpenClaw nó là một **platform architecture khá hoàn chỉnh**, chứ không phải một dự án agent chấp vá. Ranh giới giữa Gateway, sessions, transcript persistence, compaction, memory, plugins, channels và models đã được tải liệu hóa thành các khái niệm riêng. Cách tách sessions.json khỏi transcript JSONL, việc phân biệt **compaction** với **session pruning**, hay việc plugin có thể đăng ký channels, tools, providers, hooks, routes và services cho thấy team đã nghĩ khá nghiêm túc về scale và extensibility.

- **Kiến trúc phân lớp rõ**

Gateway, session persistence, memory, plugins, channels và models đều có docs riêng. Đây là dấu hiệu tốt vì code thường dễ bảo trì hơn khi khái niệm hệ thống đã được formalize.

- **Khả năng mở rộng rất cao**

Plugin API không chỉ thêm tool mà còn thêm channel, provider, CLI, route, service, hook, context engine và memory prompt section. Điều này biến OpenClaw thành một nền tảng có thể được mở rộng theo nhiều hướng.

- **Context lifecycle được thiết kế kỹ**

Việc dùng transcript append-only, compaction persisted, pruning transient và pre-compaction memory flush cho thấy bài toán “session dài hạn” đã được xử lý khá bài bản. Đây là điểm mạnh kỹ thuật đáng kể so với nhiều agent app đơn giản hơn.

- **Model/provider abstraction tốt**

Hệ thống có auth profiles, provider catalogs, model scanning và failover nhiều tầng. Điều này giúp runtime bền hơn khi gặp rate limit, token expiry hay outage từ provider.

- **Channel abstraction có contract tương đối rõ**

Docs plugin nói core giữ shared message host và execution dispatch, còn channel plugins giữ phần adapter đặc thù của từng nền tảng. Đây là hướng thiết kế tốt để tránh if-else theo nền tảng lan khắp core.

- **Hệ docs tốt, hỗ trợ đọc code**

Một điểm mạnh rất thực dụng là docs đủ sâu để hiểu được các subsystem lớn trước khi phải đào vào source cụ thể. Với codebase lớn, điều này rất có giá trị cho maintainability và onboarding.

4. Chất lượng code: điểm yếu & nợ kỹ thuật

Điểm yếu lớn nhất của OpenClaw là **session/state complexity**. Khi một runtime phải đồng thời quản lý transcript append-only, compaction ghi vào lịch sử, pruning chỉ áp dụng tạm cho prompt, semantic memory retrieval, session mapping theo channel/binding, và cho nhiều surfaces cùng truy cập một Gateway trung tâm, thì vùng state machine này sẽ rất dễ phát sinh edge cases. Thêm vào đó, OpenClaw có surface area rất lớn: channels, browser control, provider failover, plugin ecosystem, UI surfaces, daemon lifecycle. Điều này giúp sản phẩm mạnh, nhưng cũng đồng nghĩa với nợ kỹ thuật cao hơn và việc hardening đồng bộ sẽ khó hơn nhiều.

- **Session engine rất phức tạp**
sessions.json, transcript JSONL, compaction, pruning, memory flush, retrieval và routing đều giao nhau trong cùng vòng đời session. Đây gần như chắc chắn là vùng khó nhất để debug và ổn định hóa.
- **Stateful gateway là lợi thế nhưng cũng là rủi ro**
Khi Gateway là source of truth cho tất cả sessions và channels, mọi vấn đề về lifecycle, recovery hay concurrency ở đó đều có tác động toàn hệ thống.
- **Surface area quá rộng**
OpenClaw không chỉ là agent core; nó còn chạm vào nhiều channel adapters, models/providers, browser/system interactions, dashboard, daemon install và plugin ecosystem. Bề mặt rộng khiến testing và hardening khó đồng đều.
- **Plugin openness làm tăng rủi ro consistency**
Vì plugin có thể mở rộng rất nhiều lớp của runtime, chất lượng toàn hệ thống phụ thuộc không nhỏ vào chất lượng plugin authorship, testing discipline và mức độ ổn định của contracts giữa core với ecosystem.
- **Skills/plugins khó giữ chuẩn đồng nhất**
Việc tách “tool capability” và “skill guidance” rất mạnh về mở rộng, nhưng cũng có mặt trái: hành vi thực tế của agent có thể phụ thuộc lớn vào chất lượng prompt/skill chứ không chỉ vào code core. Điều này dễ tạo ra sự không đồng đều.
- **Độ khó đọc repo cao**
Dù repo được chia subsystem khá rõ, codebase vẫn lớn và đa bề mặt. Nếu chưa hiểu contracts về sessions, plugins, models và channels trước, rất dễ bị lạc khi đọc source trực tiếp.

TRIỂN KHAI LẠI OPENCLAW BẰNG RUST (IRON CLAW)

IronClaw một bản triển khai lại lấy cảm hứng từ OpenClaw nhưng viết bằng Rust, thay vì dùng stack Node.js/TypeScript như OpenClaw gốc. Repo phổ biến nhất mô tả rất rõ: **“IronClaw is a Rust reimplementation inspired by OpenClaw.”**

Nói dễ hiểu hơn:

- **OpenClaw gốc:** agent runtime/gateway viết chủ yếu theo stack Node.js/TypeScript.
- **IronClaw:** làm lại ý tưởng đó bằng Rust, thường nhấn mạnh hơn vào **privacy, security, performance**.
- Không phải chỉ là plugin hay wrapper cho OpenClaw
- Mà là **một implementation khác**, dùng ngôn ngữ và kiến trúc runtime khác

- Nhưng vẫn **lấy cảm hứng từ mô hình của OpenClaw**: personal AI assistant, nhiều channel, dùng tools, chạy local/self-hosted.

Khiến khai lại bằng Rust vì

- kiểm soát bộ nhớ tốt hơn
- tăng an toàn runtime
- hiệu năng cao hơn
- giảm một số rủi ro vốn hay gặp trong hệ sinh thái scripting/runtime động

Những điểm khác nhau giữa OpenClaw và IronClaw

1. OpenClaw là gateway-centric, còn IronClaw có vẻ runtime/module-centric hơn

OpenClaw docs nhấn mạnh rất mạnh khái niệm **Gateway**: một tiến trình trung tâm giữ sessions, routing và channel connections cho toàn hệ thống. Kiến trúc của nó xoay quanh một **single long-lived Gateway**.

IronClaw thì từ repo và docs.rs cho thấy cấu trúc nghiêng về các **crate/module chức năng** như skills, tools, worker, transcription, webhooks, tunnel, timezone, memory/workspace, và còn có cả ironclaw_safety crate riêng. Điều này gợi ý IronClaw không copy nguyên xi mô hình “Gateway là trung tâm tuyệt đối” của OpenClaw, mà đẩy mạnh hơn vào **module hóa theo capability** và **safety/runtime crates**.

2. OpenClaw nhấn vào session engine, IronClaw nhấn vào safety layer

OpenClaw docs đi rất sâu vào **session management, compaction, JSONL transcript history**. Compaction được lưu ngược vào session history, cho thấy session engine là lõi rất quan trọng của OpenClaw.

IronClaw repo lại lộ ra những dấu hiệu khác: có crate tên **ironclaw_safety**, có **fuzz** directory, có benchmark cho “safety layer hot paths”, có commit notes về hardening production container và patch vulnerability. Nghĩa là ngoài chuyện dùng Rust, repo IronClaw cho thấy họ ưu tiên **safety/security engineering** như một subsystem riêng, chứ không chỉ coi đó là hệ quả tự nhiên của việc dùng Rust.

3. OpenClaw mở rộng chủ yếu bằng plugins/channels/providers, còn IronClaw có xu hướng registry/manifests/WASM

OpenClaw docs mô tả rất rõ plugin architecture: channel plugins, tool plugins, provider plugins, shared message tool, action adapters.

IronClaw repo thì cho thấy những hướng kỹ thuật khác:

- có thư mục **registry**
 - có **providers.json**
 - commit note “move MCP servers from code to JSON manifests”
 - có thư mục **wit** với ghi chú “WASM channel attachments...”
- Những dấu hiệu này cho thấy IronClaw có xu hướng formalize integration bằng **manifest/registry/WASM-oriented contracts** nhiều hơn, thay vì chỉ dựa vào plugin SDK kiểu OpenClaw.

4. IronClaw có vẻ đầu tư mạnh hơn vào worker / containerized execution

Docs.rs của IronClaw có module **worker** với mô tả “Worker mode for running inside Docker containers.” Repo cũng có Dockerfile.worker. Điều này cho thấy IronClaw xem **worker/container execution** là một mode quan trọng của runtime.

OpenClaw thì có self-hosting và nhiều channel/runtime surfaces, nhưng từ docs bạn thấy trọng tâm của nó vẫn là **gateway + sessions + channels**, chứ không lộ ra một kiến trúc workerized/container-centric rõ như IronClaw.

5. IronClaw có vẻ thiên về privacy/security-first productization

Repo IronClaw và site của họ đều nhấn mạnh “secure personal AI assistant”, “peace of mind”, encrypted enclaves/secure alternative, trong khi OpenClaw thiên về capability, channels, skills và agent execution. Dù đây là positioning, nó cũng phản ánh phần nào định hướng codebase: IronClaw có nhiều dấu hiệu về hardening, coverage gates, fuzzing, safety crate, dependency vulnerability patching.

NEMOCLAW

1. Kiến trúc tổng thể

NemoClaw không phải một agent runtime hoàn toàn độc lập như OpenClaw, mà là một open-source reference stack để chạy OpenClaw an toàn hơn trong NVIDIA OpenShell. Tài liệu kiến trúc của NVIDIA nói rất rõ NemoClaw có hai thành phần chính: một TypeScript plugin tích hợp với OpenClaw CLI/gateway, và một Python blueprint dùng để điều phối các tài nguyên OpenShell. Nói cách khác, agent runtime cốt lõi vẫn là OpenClaw, còn NemoClaw thêm một lớp sandboxing, policy control, inference routing, và lifecycle orchestration để đưa agent vào môi trường an toàn và có kiểm soát hơn.

- **Reference stack, không phải agent core mới**
README và docs đều mô tả NemoClaw là một stack giúp “run OpenClaw always-on assistants more safely,” chứ không tuyên bố thay thế OpenClaw bằng một runtime agent khác.
- **Hai thành phần lõi**
Kiến trúc chính thức gồm:
 - **TypeScript plugin** chạy cùng OpenClaw trong sandbox
 - **Python blueprint** để resolve, verify và apply sandbox resources trong OpenShell
- **OpenShell là lớp thực thi bảo mật**
NemoClaw cài OpenShell runtime, một phần của NVIDIA Agent Toolkit, để tạo môi trường chạy agent an toàn hơn với policy-based controls.
- **Kiến trúc xoay quanh sandbox + policy**
Tài liệu “How NemoClaw Works” mô tả NemoClaw dùng CLI + blueprint để đưa OpenClaw vào một sandbox được kiểm soát, nơi network, filesystem và inference có thể bị ràng buộc bởi policy khai báo.
- **Inference và model nằm trong stack triển khai**
NemoClaw tích hợp với inference configuration và các model như NVIDIA Nemotron, nhưng mục tiêu của nó là cung cấp cách chạy agent an toàn và dễ quản trị hơn, chứ không phải đổi bản chất agent loop của OpenClaw.

Commented [NM1]: chưa hiểu khúc này

Commented [NM2R1]: @Hai Nguyen Nhat

2. Cách tổ chức mã nguồn

Ở mức repo, NemoClaw được tổ chức khá gọn và rõ ràng hơn OpenClaw. Repo chính thức có các thư mục đáng chú ý như nemoclaw/, nemoclaw-blueprint/, docs/, test/, ci/, bin/, cùng cả package.json lẫn pyproject.toml. Chỉ riêng dấu hiệu này đã cho thấy đây là một codebase **đa ngôn ngữ**: phần plugin/CLI nghiêng về Node/TypeScript, còn phần blueprint/orchestration nghiêng về Python. Tài liệu kiến trúc của NVIDIA cũng xác nhận đúng cấu trúc này khi mô tả plugin TypeScript và blueprint Python như hai khối chính của hệ thống.

- **nemoclaw/ là phần plugin/CLI**
Docs kiến trúc nói plugin là một package TypeScript mỏng, đăng ký inference provider và slash command /nemoclaw, chạy in-process với OpenClaw gateway bên trong sandbox.
- **nemoclaw-blueprint/ là phần orchestration**
Đây là nơi blueprint Python sống, chịu trách nhiệm resolve artifact, verify digest, lập kế hoạch tài nguyên và apply sandbox qua OpenShell.
- **Repo dùng cả hệ sinh thái Node và Python**
Việc xuất hiện đồng thời package.json, pyproject.toml, uv.lock, vitest.config.ts cho thấy project chia phần plugin/tooling theo Node/TS và phần orchestration theo Python.
- **Có test và CI riêng**
Repo có test/ và ci/, đồng thời lịch sử commit gần đây cho thấy họ đang tăng coverage thresholds và thêm comprehensive unit tests. Điều này gợi ý NVIDIA đang cố harden codebase khá sớm dù dự án còn alpha.
- **Docs được xem như thành phần chính của dự án**
Repo và docs.nvidia.com có đầy đủ các phần overview, architecture, how-it-works, commands, quickstart. Với một stack orchestration/sandbox như NemoClaw, tài liệu đóng vai trò rất lớn vì nhiều giá trị của hệ thống nằm ở lifecycle và policy, không chỉ ở business logic trong code.

3. Chất lượng code: điểm mạnh

Điểm mạnh lớn nhất của NemoClaw là **ranh giới kiến trúc rất rõ**. NVIDIA không cố nhét toàn bộ logic vào một runtime duy nhất, mà tách rõ **plugin layer** và **blueprint/orchestration layer**. Plugin chịu trách nhiệm gắn vào OpenClaw và expose command/provider cần thiết; blueprint chịu trách nhiệm lifecycle của sandbox và tài nguyên OpenShell. Với một hệ thống thiên về security/orchestration, việc formalize ranh giới như vậy là một lựa chọn kỹ thuật tốt vì giúp giảm coupling và dễ harden từng lớp độc lập.

- **Kiến trúc phân vai rõ ràng**
Plugin TypeScript là lớp tích hợp với OpenClaw, còn blueprint Python là lớp điều phối sandbox. Đây là dấu hiệu tốt vì trách nhiệm của từng phần được chia tương đối mạch lạc.
- **Thiết kế đúng bài toán enterprise hardening**
NemoClaw tập trung vào OpenShell sandbox, declarative policy và managed inference thay vì viết lại agent runtime. Điều này hợp lý vì pain point chính của doanh nghiệp nằm ở security boundary và governance, không phải ở việc cần thêm một loop agent nữa.
- **Policy-first design**
README và docs nhấn mạnh network requests, file access và inference calls đều có thể được kiểm soát bằng policy. Đây là một ưu điểm lớn nếu mục tiêu là đưa agent vào môi trường doanh nghiệp hoặc data-sensitive workflows.
- **Hướng triển khai thực dụng**
NemoClaw cung cấp CLI làm entrypoint chính, quickstart rõ ràng, commands cho

launch/connect/status/logs/deploy. Kiểu thiết kế này tốt cho operability vì người vận hành có một mặt tiền tương đối nhất quán để quản sandboxed agents.

- **Repo có dấu hiệu hardening sớm**
Lịch sử commit hiển thị nhiều thay đổi liên quan tới security và quality như validate endpoint URL để ngăn SSRF, stop leaking API keys in process args, build-arg code injection fix, thêm unit tests và coverage thresholds. Dù đây chưa phải bằng chứng code đã “chín”, nó là dấu hiệu NVIDIA đang rất chú ý tới secure-by-default và quality gates ngay từ giai đoạn đầu.
- **Hardware-agnostic ở mức platform**
Docs NVIDIA nói NemoClaw là stack open-source có thể chạy trên nhiều môi trường, còn trang docs hub mô tả use cases từ cloud, on-prem, RTX PCs tới DGX Spark. Điều này cho thấy thiết kế không bị khóa cứng vào một kiểu deployment duy nhất.

4. Chất lượng code: điểm yếu & nợ kỹ thuật

Điểm yếu của NemoClaw hiện tại là **độ chín của dự án còn thấp**. README chính thức ghi rất rõ đây là **alpha software**, early preview từ 16/03/2026, chưa production-ready và interface/API/hành vi có thể thay đổi mà không báo trước. Nghĩa là dù kiến trúc có vẻ đúng hướng, codebase vẫn còn ở giai đoạn định hình contract và hoàn thiện vận hành, chưa phải mức ổn định để coi như một platform trưởng thành.

- **Dự án còn alpha, chưa production-ready**
Đây là cảnh báo chính thức từ README, nên nếu đánh giá chất lượng mã nguồn thì phải coi “mức trưởng thành thấp” là một rủi ro rất thật, không chỉ là suy đoán.
- **Phụ thuộc mạnh vào OpenClaw + OpenShell**
NemoClaw không sở hữu toàn bộ runtime; nó đứng giữa OpenClaw và OpenShell. Điều này giúp nó tập trung vào security/orchestration, nhưng cũng tạo thêm độ phức tạp ở integration boundaries và compatibility giữa các lớp.
- **Codebase đa ngôn ngữ làm tăng chi phí bảo trì**
Việc chia phần plugin bằng TypeScript và phần blueprint bằng Python là hợp lý về mặt trách nhiệm, nhưng cũng làm tăng độ phức tạp build/test/release và yêu cầu đội ngũ phải duy trì hai ecosystem toolchain khác nhau.
- **Bài toán khó nằm ở policy correctness**
Vì NemoClaw đặt trọng tâm vào sandbox, policy và access governance, phần rủi ro lớn không nằm ở “có chạy được không” mà ở “policy có đúng và kín không”. Những commit gần đây vá SSRF, rò API keys, code injection cho thấy đây đúng là vùng dễ phát sinh lỗ hổng.
- **Tính năng vận hành còn đang mở rộng**
Docs commands ghi nemoclaw deploy là experimental. Quickstart cũng cho thấy sandbox image khá lớn và còn yêu cầu fresh installation of OpenClaw. Đây là dấu hiệu rằng trải nghiệm vận hành/deploy vẫn đang trong quá trình hoàn thiện, chưa đạt độ mượt của một sản phẩm ổn định lâu năm.
- **Nợ kỹ thuật tiềm ẩn ở integration surface**
Vì NemoClaw phải điều phối CLI, blueprint, OpenShell runtime, sandbox policies, inference provider và OpenClaw gateway, nên bề mặt tích hợp của nó rộng hơn về ngoài. Đây là loại hệ thống dễ phát sinh lỗi ở boundary hơn là ở business logic đơn lẻ. Nhận định này bám trực tiếp vào kiến trúc plugin + blueprint + OpenShell mà docs mô tả.

